

Informe — Examen Final Transversal (EFT)

CI/CD con Contenedores, Autoescalado y Balanceo de Carga en AWS EKS¹ Aplicación «Tienda Perritos»

Instituto:	Duoc UC
Asignatura:	Introducción a Herramientas DevOps
Docente:	Rafael Videla
Integrantes (dupla):	Francisco Gómez Ramos · Benjamin Aravena Rosales
Fecha:	06 / 07 / 2026
Repositorio GitHub:	github.com/FranciscoGomezRa/tienda-perritos-eft
Región AWS / Clúster:	us-east-1 / tienda-eks
Namespace:	tienda

¹ Documento elaborado con apoyo de una herramienta de inteligencia artificial (Claude, de Anthropic) para la redacción y estructuración del texto. El contenido técnico, las decisiones de arquitectura y la evidencia fueron ejecutados y verificados por los integrantes.

1. Introducción

El presente informe documenta el desarrollo del Examen Final Transversal (EFT) de la asignatura Introducción a Herramientas DevOps (ISY1101), cuyo propósito es llevar la aplicación web «Tienda Perritos» —una tienda en línea de productos para mascotas compuesta por un frontend (Nginx + JavaScript), un backend (Node.js / Express) y una base de datos MySQL— desde el desarrollo local contenedorizado hasta un entorno de orquestación de contenedores productivo en la nube.

Para lograrlo se implementó: un entorno de desarrollo local con Docker Compose; imágenes de contenedor minimalistas (Dockerfile multietapa, usuario sin privilegios); un pipeline de integración y despliegue continuo (CI/CD) con GitHub Actions que ejecuta el ciclo test → build → escaneo → push → deploy; un registro privado de imágenes en Amazon ECR; un clúster de Kubernetes gestionado en AWS EKS con autoescalado horizontal de réplicas (HPA) y balanceo de carga mediante un Elastic Load Balancer; gestión segura de credenciales; observabilidad; y controles de seguridad (escaneo de vulnerabilidades con Trivy y aislamiento de red de la base de datos con una NetworkPolicy).

El informe se organiza en los nueve puntos que el encargo exige justificar (sección 4). Cada punto presenta la implementación realizada, su justificación técnica y la evidencia gráfica real capturada durante la ejecución del proyecto, con los comandos de CloudShell asociados, de modo que cada criterio de la rúbrica quede respaldado por una prueba verificable.

2. Contexto del Caso

Tienda Perritos es un comercio electrónico dedicado a la venta de productos para mascotas. Tras contenedorizar su aplicación con Docker, el negocio enfrenta un desafío: su catálogo y su tráfico crecen, las campañas y los picos de demanda son cada vez más frecuentes, y una caída del sitio o un despliegue manual fallido se traducen directamente en ventas perdidas.

Frente a este escenario, el equipo técnico debe evolucionar desde «contenedores que corren en una máquina» hacia una plataforma de orquestación productiva capaz de:

- ejecutar la aplicación de forma escalable, agregando o quitando réplicas según la carga real;
- mantenerse altamente disponible y tolerante a fallos, repartiéndose en varias zonas de disponibilidad y autorrecuperándose ante la caída de un contenedor;
- automatizar por completo los despliegues desde GitHub, con pruebas automáticas y escaneo de seguridad antes de publicar, eliminando pasos manuales propensos a error;
- registrar logs y métricas para diagnosticar problemas y medir el desempeño;

- y proteger las credenciales sensibles (acceso a AWS y a la base de datos) fuera del código fuente.

Este informe responde a ese contexto: presenta la arquitectura diseñada para Tienda Perritos sobre AWS EKS y demuestra, con evidencia, que la solución cumple cada uno de esos objetivos de negocio.

3. Propuesta de Solución

La solución orquesta los tres servicios de Tienda Perritos en un clúster Amazon EKS (tienda-eks, región us-east-1, namespace tienda), desplegando las imágenes desde un registro privado Amazon ECR y automatizando todo el ciclo de vida con GitHub Actions. Sus componentes principales son:

- **Desarrollo local (Docker Compose):** un archivo docker-compose.yml levanta los 3 contenedores (frontend, backend, db) con red interna, healthcheck de MySQL y volumen local. Los servicios usan los mismos nombres DNS que en Kubernetes, por lo que el mismo artefacto corre sin cambios en local y en la nube.
- **CI/CD (GitHub Actions):** pipeline de dos etapas — un job de **test** (Jest + supertest) que actúa como compuerta y, si pasa, un job **build-and-deploy** que construye las imágenes, las escanea con Trivy, las publica en ECR y despliega en EKS.
- **Registro de imágenes (Amazon ECR):** tres repositorios privados (tienda-frontend, tienda-backend, tienda-db) con tags SHA corto + latest para trazabilidad.
- **Red (VPC):** tienda-vpc (10.0.0.0/16) en 2 zonas de disponibilidad; 6 subredes: 2 públicas (Load Balancer y NAT Gateway) y 4 privadas (nodos worker sin IP pública).
- **Cómputo y orquestación (EKS):** plano de control gestionado por AWS y un node group de instancias t3.medium (2/2/4) en las subredes privadas; tres Deployments con requests/limits y sondas de salud; HPA sobre frontend y backend alimentado por Metrics Server.
- **Conectividad, balanceo y seguridad de red:** Service tienda-frontend de tipo LoadBalancer (Classic ELB público); services internos tienda-backend y tienda-db por DNS interno; NetworkPolicy que aísla la base de datos (solo el backend alcanza el puerto 3306).

Flujo de la petición del usuario: Usuario → ELB (subred pública) → Service/Pods frontend (Nginx) → Service backend (DNS interno tienda-backend) → Pods backend (Node/Express) → Service db (tienda-db) → MySQL.

Flujo CI/CD: commit a main → GitHub Actions (job test → job build-and-deploy con Environment production) → build + escaneo Trivy → push a ECR → kubectl set image + rollout status sobre EKS. Los secretos viven en un GitHub Environment, nunca en el repositorio.

Arquitectura — CI/CD + AWS EKS + HPA · Tienda Perritos

Patrón de producción · us-east-1 · cuenta 618629205592 · clúster tienda-eks

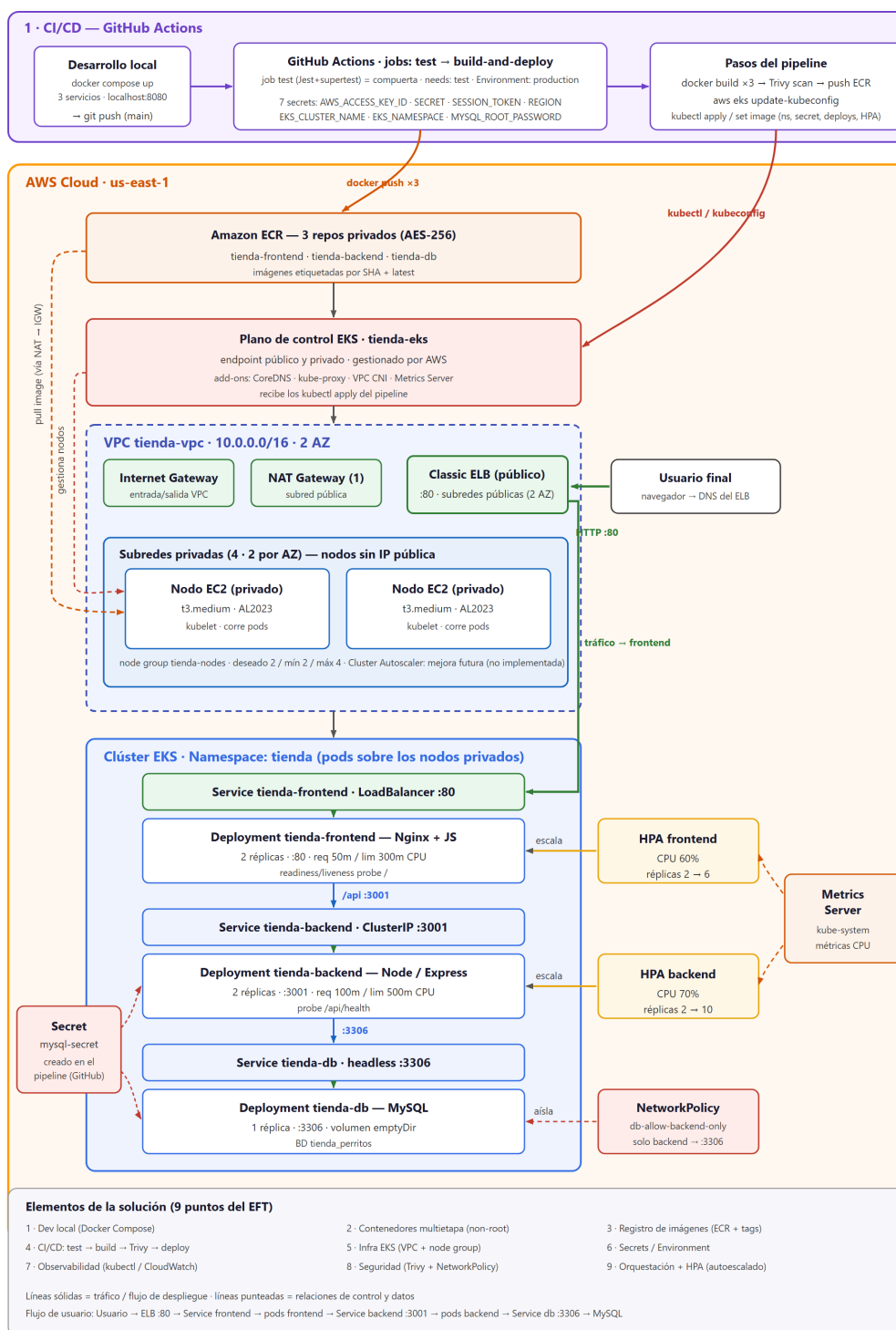


Figura. Diagrama de arquitectura de la solución: desarrollo local con Compose, pipeline CI/CD con test y Trivy, e infraestructura AWS EKS.

4. Desarrollo de la Solución — los 9 puntos

A continuación se desarrolla cada uno de los nueve puntos que el encargo exige justificar, con su implementación, justificación técnica y evidencia.

Punto 1 · Método de integración (frontend ↔ backend ↔ base de datos)

La aplicación es de tres capas y la comunicación entre ellas se resuelve por nombre, no por IP, tanto en local como en la nube:

- **Frontend → Backend:** el frontend (Nginx) sirve la interfaz y actúa como proxy inverso. Toda petición a la ruta /api/ se reenvía al backend por su nombre de servicio (tienda-backend) en el puerto 3001; el navegador solo habla con el frontend.
- **Backend → Base de datos:** el backend (Node/Express) mantiene un pool de conexiones a MySQL, apuntando al host tienda-db en el puerto 3306, con credenciales tomadas de variables de entorno (DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, DB_PORT).
- **Resolución de nombres:** en Kubernetes los nombres los resuelve CoreDNS; en Docker Compose, la red interna de Docker. Como los nombres son idénticos en ambos entornos, el código no distingue dónde corre.

Flujo completo de una petición: Usuario → DNS del ELB :80 → Service frontend → pods Nginx → (proxy /api) → Service backend :3001 → pods Node/Express → Service db :3306 → MySQL. El frontend es la única cara pública; backend y base de datos permanecen internos.

Evidencia

The screenshot displays a web application titled "Tienda de Alimentos para Perritos" with a subtitle "Gestión CRUD de productos en contenedores Docker (Frontend + Backend + MySQL)". The interface includes a "Productos" section with a "Cargar Productos" button and a table listing items. Below the table is a "Nuevo producto" form with fields for "Nombre", "Descripción", "Precio", and "Stock", along with "Guardar" and "Cancelar" buttons. A green message "Producto creado correctamente." is visible at the bottom of the form.

ID	Nombre	Descripción	Precio	Stock	Acciones
6	NUEVO PRODUCTO PÁPI	POLLITO	\$1313.00	2	<button>Editar</button> <button>Eliminar</button>
5	Bravery pollo Adulto raza pequena	Sabor a pollo	\$25990.00	23	<button>Editar</button> <button>Eliminar</button>
4	Alimento Adulto Pedigree	Sabor carne	\$15990.00	40	<button>Editar</button> <button>Eliminar</button>
3	Snacks Dentales	Ayuda a la limpieza dental	\$5990.00	30	<button>Editar</button> <button>Eliminar</button>

Nuevo producto

Nombre:

Descripción:

Precio:

Stock:

Guardar Cancelar

Producto creado correctamente.

Figura. CRUD funcional en la web (alta/edición/baja de productos): comunicación Front→Back→DB operativa de extremo a extremo.

Punto 2 · Contenedores (imágenes, redes y orquestación local con Docker Compose)

Cada componente tiene su propia imagen de contenedor, construida para ser pequeña y segura, y los tres se orquestan localmente con Docker Compose antes de llevarlos a la nube.

Imágenes por componente (minimalistas)

- **Backend — Dockerfile multietapa:** la etapa 1 (deps) instala solo dependencias de producción con `npm ci --omit=dev` (las `devDependencies` como `jest` y `supertest` nunca entran a la imagen final); la etapa 2 (runtime) es `node:18-alpine` con solo `node_modules` de producción y el código, `NODE_ENV=production` y endurecimiento con `USER node` (el proceso no corre como `root`). Beneficio: imagen más liviana, menor superficie de ataque y arranque más rápido.
- **Frontend:** imagen `nginx:alpine` con los estáticos y la configuración de proxy.
- **Base de datos:** imagen basada en MySQL con el esquema y los datos semilla.
- **.dockerignore** en frontend, backend y db para no copiar `node_modules`, `.env`, `.git` ni documentación a las imágenes.

Orquestación local (docker-compose.yml)

- 3 servicios: `tienda-db`, `tienda-backend`, `tienda-frontend`, sobre la red interna «perritos» (los servicios se resuelven por nombre).
- Healthcheck de MySQL (`mysqladmin ping`): el backend arranca solo cuando la BD está lista (`depends_on: condition: service_healthy`).
- Volumen local `db_data` para la persistencia de la base en desarrollo.
- Frontend publicado en `http://localhost:8080` (`8080:80`).

Decisión de diseño clave: los servicios del compose usan los **mismos nombres DNS** que los Services de Kubernetes, por lo que el mismo artefacto corre sin cambios en local y en producción.

Rol de Compose frente a ECR y EKS (no compiten, se complementan): Docker Compose orquesta el entorno de desarrollo local (PC); Amazon ECR es el registro de imágenes en la nube (la «bodega» con tags); Amazon EKS es la orquestación de producción (escala, balancea, autorrecupera). Compose es el primer eslabón de la cadena: dev local (compose) → push a GitHub → pipeline (test → build → scan) → publica en ECR → despliega en EKS.

Evidencia

examenfinal				0.5%	1 hour ago	
tienda-db-1	025f581d9127	examenfinal-tienda-db		0.5%	1 hour ago	
tienda-backend-1	2395093d54b7	examenfinal-tienda-backend	\$001.3001	0%	1 hour ago	
tienda-frontend-1	140580530e7c	examenfinal-tienda-frontend	\$080.80	0%	1 hour ago	

Figura. docker compose ps con los 3 servicios Up y la base de datos en estado healthy.

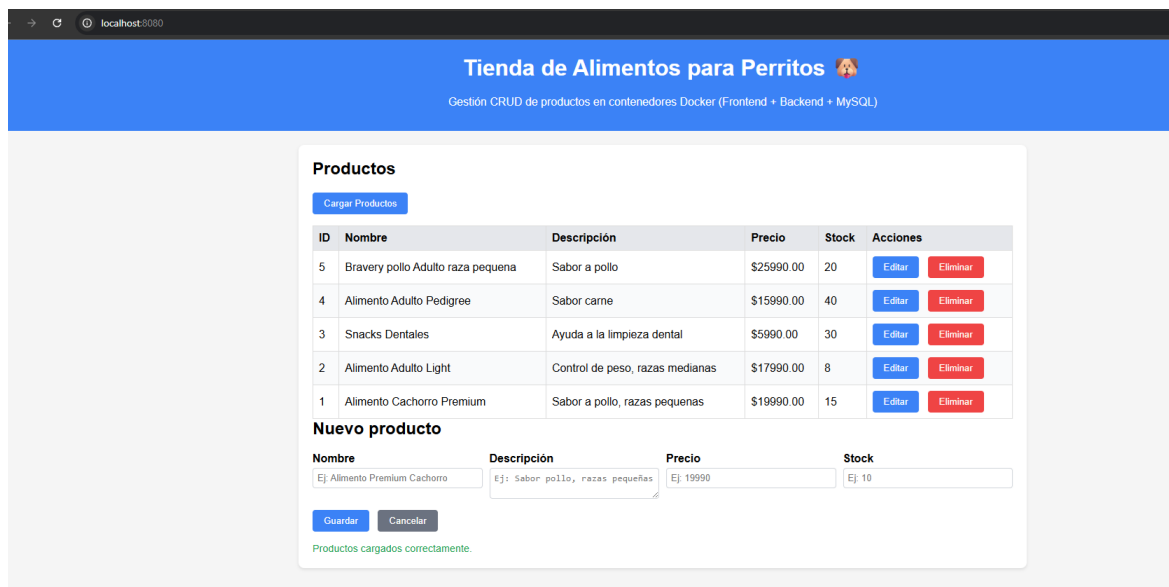


Figura. Aplicación con productos funcionando en el entorno local (http://localhost:8080).

Punto 3 · Registro de imágenes (Amazon ECR + tags para trazabilidad)

Las imágenes construidas por el pipeline se publican en Amazon ECR, el registro privado de contenedores de AWS. Se crearon tres repositorios privados —tienda-frontend, tienda-backend y tienda-db— con mutabilidad de tags Mutable, cifrado AES-256, en us-east-1. (La alternativa según la pauta sería Docker Hub; se eligió ECR por integrarse nativamente con EKS y con las credenciales del pipeline.)

Etiquetado para trazabilidad: cada imagen se publica con dos tags: `<sha7>` —los 7 primeros caracteres del commit (`IMAGE_TAG=${GITHUB_SHA::7}`), inmutable y trazable exactamente al commit que la generó— y `:latest`, puntero a la última versión. El despliegue usa el tag `<sha7>` (`kubectl set image`), garantizando que EKS corre la imagen exacta de ese commit.

Qué se define en ECR vs en EKS: en ECR, los repositorios y su configuración (privado/público, mutabilidad, cifrado, región): solo almacena imágenes versionadas. En EKS, la infraestructura y la ejecución (clúster, nodos y manifiestos). Regla: si es una imagen → ECR; si es infraestructura o ejecución → EKS.

Evidencia

The screenshot shows the Amazon Elastic Container Registry (ECR) console. A green notification bar at the top states 'Repositorio privado creado correctamente, tienda-db'. Below this, the 'Repositorios privados (3)' section is visible. A search bar is present with the placeholder 'Buscar por subcadena de repositorio'. A table lists the three repositories:

Nombre del repositorio	URI	Creado en	Inmutabilidad de etiqueta	Tipo de cifrado
tienda-backend	268450856324.dkr.ecr.us-east-1.amazonaws.com/tienda-backend	28 de junio de 2026, 13:25:52 (UTC-04)	Mutable	AES-256
tienda-db	268450856324.dkr.ecr.us-east-1.amazonaws.com/tienda-db	28 de junio de 2026, 13:26:14 (UTC-04)	Mutable	AES-256
tienda-frontend	268450856324.dkr.ecr.us-east-1.amazonaws.com/tienda-frontend	28 de junio de 2026, 13:25:33 (UTC-04)	Mutable	AES-256

Figura. Los 3 repositorios privados en Amazon ECR (frontend, backend y db).

Punto 4 · CI/CD (pipeline test → build → escaneo → push → deploy)

El workflow `.github/workflows/deploy-eks.yml` se dispara con cada push a main (y manualmente con `workflow_dispatch`). Está compuesto por dos jobs:

Job 1 — test (compuerta de calidad, sin secretos)

Corre en un runner `ubuntu-latest` con Node.js 18. Ejecuta `npm ci` y `npm test` (Jest + supertest). Los tests unitarios del backend mockean la base de datos, por lo que no necesitan infraestructura. Si un test falla, el despliegue ni siquiera parte, porque el segundo job declara `needs: test`.

Job 2 — build-and-deploy (con Environment production)

Solo arranca si el job test pasó. Declara `environment: production`, por lo que solo aquí se exponen los 7 secretos. Sus etapas:

- Configurar credenciales AWS (incluye el session token temporal del laboratorio).
- Login en ECR y build de las 3 imágenes con tags `:<sha7>` y `:latest`.
- **Escaneo Trivy** de las 3 imágenes (severidades HIGH y CRITICAL; ver Punto 8).
- Push de las imágenes a ECR.
- `update-kubeconfig` hacia el clúster `tienda-eks`.
- Aplicar namespace y generar el Secret de MySQL en caliente (ver Punto 6).
- Desplegar `db` → `backend` → `frontend` (`kubectl apply` + `set image` + `rollout status`).
- Verificar la Metrics API y aplicar los HPA; listar pods, servicios y HPA (evidencia en el log).

El despliegue es un **rolling update**: salen pods nuevos y recién entonces mueren los viejos, sin caída del servicio. Una corrida completa toma ~2 minutos.

Evidencia


```
PASS __tests__/api.test.js
  GET /api/health
    ✓ responde 200 con status ok (probe de Kubernetes) (21 ms)
  GET /api/productos
    ✓ devuelve la lista de productos (5 ms)
    ✓ responde 500 si la BD falla (15 ms)
  GET /api/productos/:id
    ✓ responde 404 si el producto no existe (4 ms)
  POST /api/productos
    ✓ responde 400 si faltan campos obligatorios (16 ms)
    ✓ crea un producto válido y responde 201 (4 ms)
  DELETE /api/productos/:id
    ✓ responde 404 si no hay filas afectadas (3 ms)

Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total
Snapshots:   0 total
Time:        0.712 s, estimated 3 s
Ran all test suites.
PS C:\Duoc\5to Semestre\DevOps\Examen Final\backend> comando npm test
```

Figura. npm test con los 7 tests verdes (la BD se mockea; son la primera compuerta del pipeline).

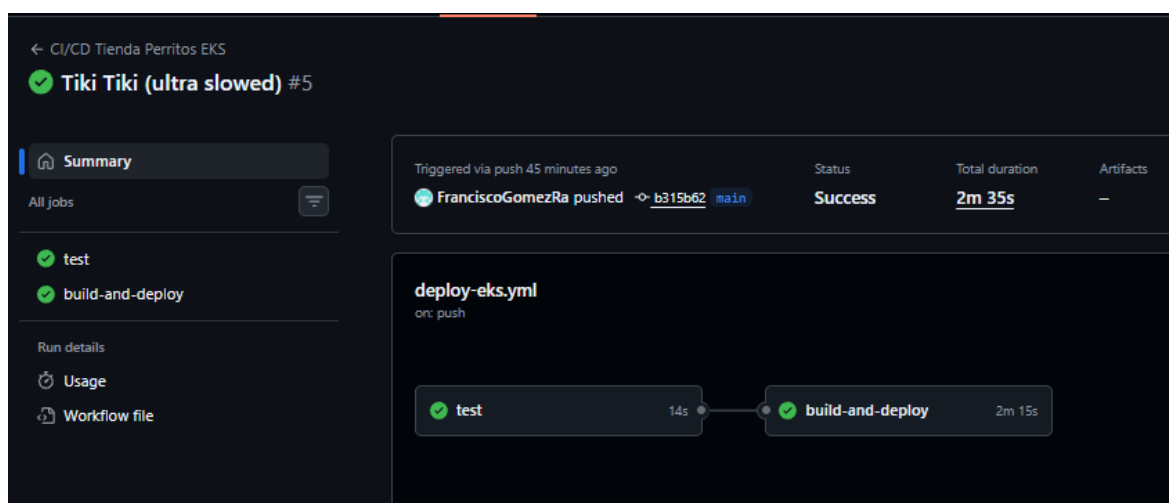


Figura. Pipeline de GitHub Actions en verde: job test + job build-and-deploy (build, escaneo Trivy, push y deploy) completados con éxito.

Punto 5 · Infraestructura en la nube (VPC, subredes, EKS y nodos)

Se creó la VPC tienda-vpc (10.0.0.0/16) con 6 subredes en 2 AZ (2 públicas + 4 privadas) y un NAT Gateway para la salida a Internet de las subredes privadas. El clúster tienda-eks usa el rol IAM LabRole tanto para el plano de control como para los nodos, y un node group de instancias t3.medium (deseado/mín/máx = 2/2/4) ubicado en las subredes privadas.

Justificación del tipo de instancia (t3.medium)

- **Límite de IPs por nodo (factor decisivo):** el plugin Amazon VPC CNI asigna una IP de la VPC a cada pod, y la cantidad disponible depende del tamaño de la instancia. t2.micro/nano permiten ~4 pods por nodo; t3.medium permite ~17 (Max IPs: 18). Con 4 IPs no alcanza ni para los pods de sistema.
- **Pods de sistema:** cada nodo ya ejecuta CoreDNS, kube-proxy, aws-node (CNI) y Metrics Server, lo que consumiría casi toda la capacidad de una instancia menor.
- **Memoria:** 0.5–1 GiB no basta para kubelet + pods de sistema + MySQL + Node/Express; los nodos quedarían NotReady. Los 4 GiB de t3.medium dan holgura.

El control de créditos se gestionó por otras vías: desired=2, observabilidad apagada y apagado del node group (a 0) al terminar cada sesión.

Permisos IAM (rol LabRole)

No fue necesario crear ni adjuntar políticas: el Learner Lab entrega LabRole con un conjunto amplio de políticas administradas (y bloquea editar roles IAM). Las 3 políticas clave para los nodos worker son AmazonEKSWorkerNodePolicy, AmazonEKS_CNI_Policy y AmazonEC2ContainerRegistryReadOnly (autoriza el docker pull desde ECR). El plano de control usa AmazonEKSClusterPolicy.

Diseño de red (patrón de producción)

Se evaluaron dos topologías. Camino A (simplificado): nodos en subredes públicas con IP pública y sin NAT — más barato, pero expone los nodos. Camino B (elegido): nodos en subredes privadas (sin IP pública), un NAT Gateway para su salida y el Load Balancer en las públicas. Se eligió B por seguridad (el único punto de entrada es el ELB), defensa en capas, coherencia (las 6 subredes cumplen un rol real) y porque es la arquitectura recomendada por AWS para EKS. El costo del NAT se mitigó usando uno solo y apagando el laboratorio al terminar.

Acceso remoto a los nodos deshabilitado (sin SSH): toda la administración se hace con kubectl (que habla con el API del clúster, no con los nodos por SSH). Habilitarlo abriría el puerto 22 y contradiría el diseño de red privada. Si excepcionalmente se necesitara una shell en un nodo, se usaría AWS SSM Session Manager (no abre el 22, funciona sobre nodos privados y queda auditado por IAM).

Relación clúster EKS ↔ ELB: el balanceador no se configura a mano: se declara con type: LoadBalancer en el frontend-service.yaml. Al aplicar el Service, el cloud-controller-manager de EKS llama a la API de AWS y aprovisiona un Classic ELB público en las subredes públicas. El ELB balancea hacia los nodos por un NodePort, y kube-proxy reenvía al pod de Nginx; si se borrara el Service, EKS eliminaría el ELB automáticamente.

URL pública (EXTERNAL-IP del ELB), HTTP puerto 80:

http://af52da2cfe9984afa87243cc593cd688-1793386799.us-east-1.elb.amazonaws.com

Evidencia

VPC > Sus VPC > Crear VPC

Crear VPC Información

Una VPC es una parte aislada de la nube de AWS que contiene objetos de AWS, como instancias de Amazon EC2. Deslizar el ratón sobre un recurso para resaltar los recursos relacionados.

Configuración de la VPC

Recursos que se van a crear Información
Cree únicamente el recurso de VPC o la VPC y otros recursos de red.

☐ Solo la VPC ☒ VPC y más

Generación automática de etiquetas de nombre Información
Ingrese un valor para la etiqueta Nombre. Este valor se utilizará para generar automáticamente etiquetas Nombre para todos los recursos de la VPC.

☒ Generar automáticamente
tienda-vpc

Bloque de CIDR IPv4 Información
Determine la IP inicial y el tamaño de la VPC mediante la notación CIDR.
10.0.0.0/16 65,536 IPs
El tamaño del bloque CIDR debe estar entre /16 y /28.

Bloque de CIDR IPv6 Información
☒ Sin bloque de CIDR IPv6
☐ Bloque de CIDR IPv6 proporcionado por Amazon

Tenencia Información
Predeterminado

► Configuración de cifrado - opcional

Vista previa

VPC Mostrar detalles
Su red virtual de AWS

Subredes (6)
Subredes dentro de esta VPC

Tablas de enrutamiento (
Dirigir el tráfico de red a los recursos

Figura. VPC tienda-vpc creada con 6 subredes (2 públicas + 4 privadas) y NAT Gateway.

Ha cambiado correctamente la configuración de la subred:

- Habilitar la asignación automática de la dirección IPv4 pública

Subredes (6) Información

Buscar subredes por atributo o etiqueta

tienda Quitar filtros

☐	Name	ID de subred	Estado	VPC	Bloquear el ...	CIDR IPv4	CIDR IPv6	ID de as
☐	tienda-vpc-subnet-private2-us-east-1b	subnet-0dddab5fd1c1e6a94	Available	vpc-0063c9fa9aef55c97 tiend...	Desactivado	10.0.144.0/20	-	-
☐	tienda-vpc-subnet-public2-us-east-1b	subnet-08f89bc3778094b74	Available	vpc-0063c9fa9aef55c97 tiend...	Desactivado	10.0.16.0/20	-	-
☐	tienda-vpc-subnet-private3-us-east-1a	subnet-021d07e9e88ca45f0	Available	vpc-0063c9fa9aef55c97 tiend...	Desactivado	10.0.160.0/20	-	-
☐	tienda-vpc-subnet-public1-us-east-1a	subnet-0d6c982e522803d76	Available	vpc-0063c9fa9aef55c97 tiend...	Desactivado	10.0.0.0/20	-	-
☐	tienda-vpc-subnet-private1-us-east-1a	subnet-0e02bec35f503cfa6	Available	vpc-0063c9fa9aef55c97 tiend...	Desactivado	10.0.128.0/20	-	-
☐	tienda-vpc-subnet-private4-us-east-1b	subnet-084d8c0ba4658d66e	Available	vpc-0063c9fa9aef55c97 tiend...	Desactivado	10.0.176.0/20	-	-

Figura. Las 6 subredes en estado Available, repartidas en us-east-1a y us-east-1b.

Acceso al punto de enlace del clúster [Información](#)

Configure el acceso al punto de enlace del servidor de la API de Kubernetes.

☐ Público

Se puede obtener acceso al punto de enlace del clúster desde fuera de la VPC. El tráfico del nodo de trabajo dejará la VPC para conectarse al punto de enlace.

☒ Público y privado

Se puede obtener acceso al punto de enlace del clúster desde fuera de la VPC. El tráfico del nodo de trabajo al punto de enlace permanecerá dentro de la VPC.

☐ Privado

Solo se puede obtener acceso al punto de enlace del clúster a través de la VPC. El tráfico desde el nodo de trabajo al punto de enlace permanecerá dentro de la VPC.

► **Configuración avanzada**

Salida del plano de control [Información](#)

Elige cómo se dirige el tráfico del plano de control de Kubernetes. En cuanto se cree un clúster, puede migrar de "Administrado por AWS" a "Enrutado por el cliente", pero no al revés.

Modo de salida | [Información](#)

☒ Administrado por AWS

El tráfico del plano de control sigue la ruta de red pública predeterminada.

☐ Enrutado por el cliente

El tráfico del plano de control se dirige a través de la VPC.

[Cancelar](#)

[Anterior](#)

[Siguiente](#)

Figura. Acceso al endpoint del clúster configurado como «Público y privado».

Paso 4

☒ **Seleccionar complementos**

Paso 5

☐ Configurar las opciones de complementos seleccionados

Paso 6

☐ Revisar y crear

☐

Agente de supervisión de nodos [Información](#)
Habilite la detección automática de problemas de estado de los nodos.
Categoría
observability
Computación compatible
EC2 y Nodos híbridos

☒

CoreDNS [Información](#)
Habilite la detección de servicios en el clúster.
Categoría
networking
Computación compatible
EC2, Nodos híbridos, Fargate, Modo automático de EKS y Clúster local

☒

kube-proxy [Información](#)
Habilite la red del servicio dentro del clúster.
Categoría
networking
Computación compatible
EC2, Nodos híbridos y Clúster local

☒

CNI de Amazon VPC [Información](#)
Habilite la red del pod dentro del clúster.
Categoría
networking
Computación compatible
EC2 y Clúster local

Figura. Complementos del clúster: CoreDNS, kube-proxy, CNI de Amazon VPC y Metrics Server.

Amazon Elastic Kubernetes Service

Clústeres > tienda-eks

Panel

Clústeres

Ajustes

- Configuración del panel
- Configuración de la consola

Amazon EKS Anywhere

- Suscripciones a Enterprise

Servicios relacionados

- Amazon ECR
- AWS Batch

Documentación

Talleres EKS

Novedad: ahora puede suscribirse a Información de contenedores de OTEL con compatibilidad con OpenTelemetry y PromQL a partir de la versión 6.2.0 del complemento Observabilidad de CloudWatch. Más información

Notifications 0 0 0 4 0

tienda-eks

Conectar

Eliminar clúster

Actualizar versión

Clúster de monitores

El soporte estándar para la versión de Kubernetes 1.35 finaliza el 26 de marzo de 2027.

Actualizar

Información del clúster

Estado

Activo

Periodo de soporte

Soporte estándar hasta el 26 de marzo de 2027

Estado del clúster

0

Problemas de estado del nodo

0

Versión de Kubernetes

1.35

Proveedor

EKS

Información sobre la actualización

4

Problemas de capacidad

0

Figura. Clúster tienda-eks en estado Active.

Informe EFT · Tienda Perritos · AWS EKS

Página 13

Establecer la configuración informática y de escalado

Configuración de computación del grupo de nodos

Estas propiedades no se pueden cambiar después de crear el grupo de nodos.

Tipo de AMI [Información](#)

Seleccione la Amazon Machine Image optimizada para EKS para los nodos.

Amazon Linux 2023 (x86_64) Estándar (AL2023_x86_64_STANDARD) ▼

Tipo de capacidad

Seleccione la opción de compra de capacidad para este grupo de nodos.

On-Demand ▼

Tipos de instancias [Información](#)

Seleccione los tipos de instancia que prefiera para este grupo de nodos.

🔍 Ingrese un tipo de instancia

t3.medium

vCPU: 2 vCPUs Memory: 4 GiB Network: Up to 5 Gigabit Max ENI: 3 Max IPs: 18 ✕

Tamaño del disco

Seleccione el tamaño del volumen de EBS adjunto para cada nodo.

20

GiB

Figura. Node group tienda-nodes: AMI AL2023, tipo t3.medium, disco 20 GiB.

Configuración de escalado del grupo de nodos

Tamaño deseado

Establezca el número deseado de nodos con los que el grupo debería lanzarse inicialmente.

2

nodos

El tamaño del nodo deseado debe ser mayor o igual que 0

Tamaño mínimo

Establezca el número mínimo de nodos al que el grupo puede escalar de forma descendente.

2

nodos

El tamaño mínimo del nodo debe ser mayor o igual que 0

Tamaño máximo

Establezca el número máximo de nodos al que el grupo puede escalar de forma ascendente.

4

nodos

El tamaño máximo del nodo debe ser mayor o igual que 1 y no puede ser inferior al tamaño mínimo

Figura. Escalado del node group: deseado / mínimo / máximo = 2 / 2 / 4.

Paso 1

Configurar grupo de nodos

Paso 2

Establecer la configuración informática y de escalado

Paso 3

Especificar redes

Paso 4

Revisar y crear

Especificar redes

Configuración de red del grupo de nodos

Estas propiedades no se pueden cambiar después de crear el grupo de nodos.

Subredes [Información](#)

Especifique las subredes de la VPC donde se ejecutarán los nodos. Para crear una nueva subred, vaya a la página correspondiente en la [Consola de VPC](#).

Seleccionar subredes

subnet-0dddab5fd1c1e6a94 | tienda-vpc-subnet-private2-us-east-1b

us-east-1b 10.0.144.0/20

subnet-021d07e9e88ca45f0 | tienda-vpc-subnet-private3-us-east-1a

us-east-1a 10.0.160.0/20

subnet-0e02bec35f503cfa6 | tienda-vpc-subnet-private1-us-east-1a

us-east-1a 10.0.128.0/20

subnet-084d8c0ba4658d66e | tienda-vpc-subnet-private4-us-east-1b

us-east-1b 10.0.176.0/20

Borrar subredes seleccionadas

☐ Configurar el acceso remoto a los nodos [Información](#)

Cancelar

Anterior

Siguiente

Figura. Node group asociado a las 4 subredes privadas; acceso remoto (SSH) en OFF.

CloudShell

us-east-1

us-east-1

us-east-1

+

```
~ $ kubectl get nodes -o wide -L topology.kubernetes.io/zone
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME	ZONE
ip-10-0-131-186.ec2.internal	Ready	<none>	97m	v1.35.6-eks-93b88c6	10.0.131.186	<none>	Amazon Linux 2023.12.20260611	6.12.90-120.164.amzn2023.x86_64	containerd://2.2.4+unknown	us-east-1a
ip-10-0-181-187.ec2.internal	Ready	<none>	99m	v1.35.6-eks-93b88c6	10.0.181.187	<none>	Amazon Linux 2023.12.20260611	6.12.90-120.164.amzn2023.x86_64	containerd://2.2.4+unknown	us-east-1b

```
~ $
```

Figura. Dos nodos Ready, uno por AZ, con EXTERNAL-IP <none> = sin IP pública.

```
kubectl get nodes -o wide -L topology.kubernetes.io/zone
```

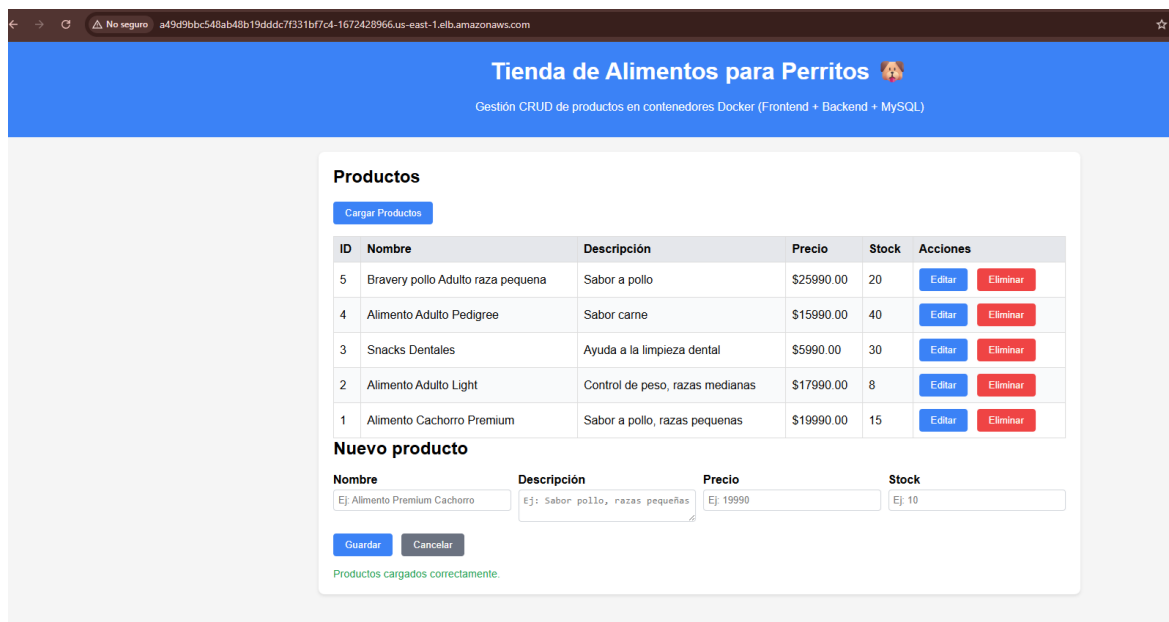



Figura. Tienda Perritos cargada en el navegador desde la URL pública del ELB.

```
aws eks update-kubeconfig --region us-east-1 --name tienda-eks
kubectl get svc -n tienda
```

Punto 6 · Configuración y secretos (GitHub Environment + mínimo privilegio)

Los secretos viven en un GitHub Environment «production» (Settings > Environments), nunca en el repositorio. El job build-and-deploy solo puede leerlos porque declara environment: production. Los 7 secrets son:

- **AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY / AWS_SESSION_TOKEN:** credenciales temporales de AWS; rotan en cada sesión del laboratorio (el token es obligatorio por ser temporales).
- **AWS_REGION** (us-east-1), **EKS_CLUSTER_NAME** (tienda-eks) y **EKS_NAMESPACE** (tienda): parámetros fijos del despliegue.
- **MYSQL_ROOT_PASSWORD:** contraseña root de MySQL, definida por el equipo; no se publica ni se versiona.

Secret de MySQL generado en el pipeline (no versionado): la contraseña de la BD nunca se escribe en el repositorio. El pipeline crea el Secret de Kubernetes en tiempo de despliegue a partir del secret de GitHub:

```
kubectl create secret generic mysql-secret \
  --from-literal=MYSQL_ROOT_PASSWORD="****" -n tienda \
  --dry-run=client -o yaml | kubectl apply -f -
```

De ese Secret lo consumen el backend (vía secretKeyRef en DB_PASSWORD) y la base de datos. El repositorio solo versiona una plantilla de referencia

(mysql-secret.example.yaml).

Mínimo privilegio y rotación: se eligió un Environment (en lugar de secrets sueltos) por su aislamiento de alcance, sus reglas de protección y su trazabilidad. Si las credenciales de AWS expiran, el pipeline falla en el paso de credenciales y basta con actualizar los 3 secrets de AWS. Nota: el Account ID (618629205592) no es un secreto sino un identificador; viaja en texto plano dentro de la URL de ECR en los manifiestos. Lo sensible son las claves de acceso, que sí van cifradas en el Environment.

Evidencia

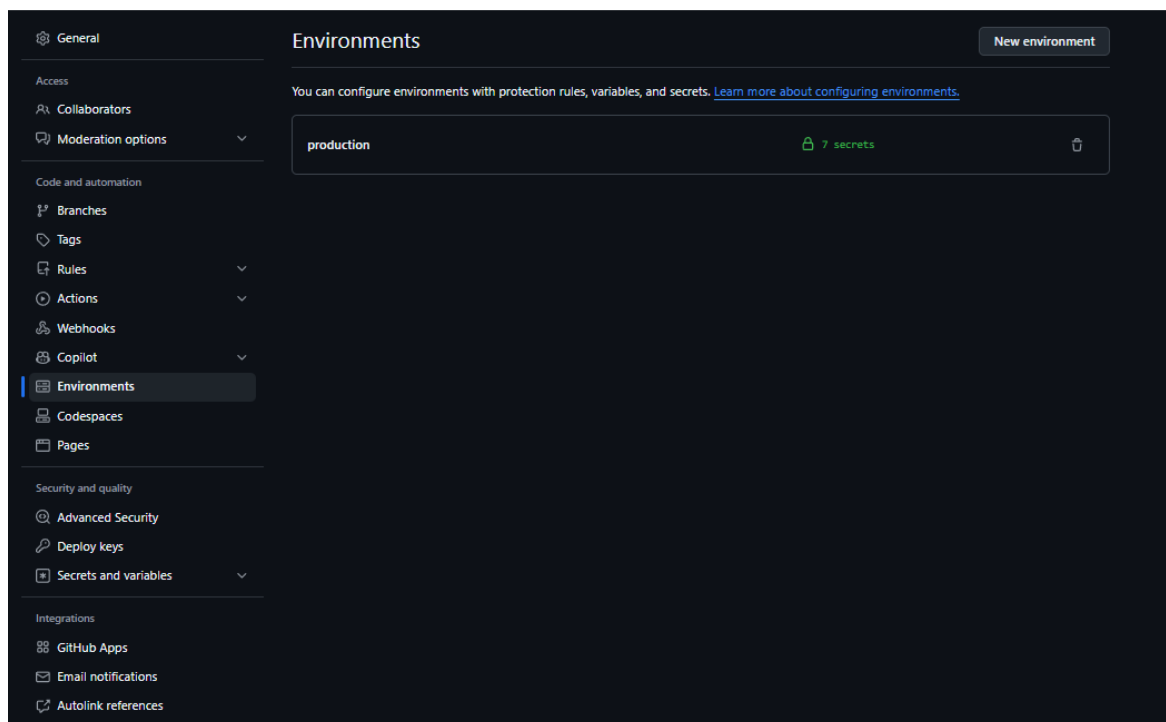


Figura. GitHub Environment «production» creado para aislar los secretos del despliegue.

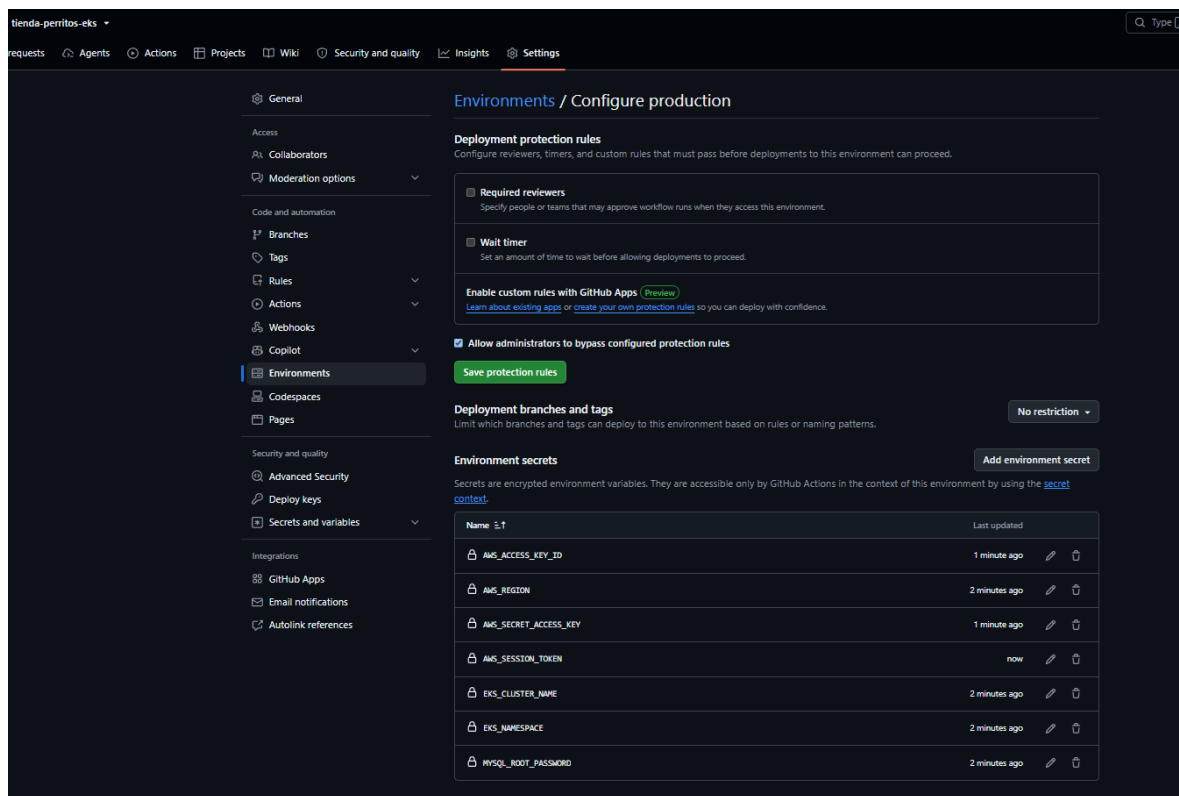


Figura. Secrets configurados en el Environment (AWS_*, EKS_*, MYSQL_ROOT_PASSWORD).

Punto 7 · Observabilidad (logs del pipeline, de la app y del plano de control)

La observabilidad se aborda en tres niveles complementarios:

- **Logs del pipeline (GitHub Actions):** cada corrida deja el detalle de test, build, escaneo, push y deploy, incluyendo el EXTERNAL-IP del ELB y el estado de pods/servicios/HPA.
- **Logs de la aplicación (nivel pod, kubectl logs):** el backend muestra «Servidor backend escuchando en puerto 3001» y «Pool de conexiones MySQL inicializado» (conexión Back↔DB correcta); el frontend (nginx) registra respuestas 200 OK, incluidas las sondas kube-probe (readiness/liveness funcionando). Las métricas de CPU/memoria por pod se ven con kubectl top (Metrics Server).
- **Logs del plano de control (nivel plataforma, CloudWatch):** se activaron temporalmente los logs del plano de control (API server + Auditoría) → log group /aws/eks/tienda-eks/cluster. El log de auditoría registra quién hizo qué en el clúster (cada kubectl apply / set image / delete). Tras la captura se desactivaron para no consumir créditos.

Distinción importante: los logs de pod responden «¿mi aplicación funciona?»; los logs de control plane responden «¿quién administró el clúster y qué operaciones ocurrieron a nivel de plataforma?». Son capas distintas. Tiempos: ~2 minutos por corrida completa del pipeline.

Evidencia

```
CloudShell

us-east-1 +

~ $ # Logs del backend (muestra el arranque + las peticiones del CRUD que hiciste)
~ $ kubectl logs deploy/tienda-backend -n tienda --tail=40
Found 2 pods, using pod/tienda-backend-848b954bd5-vcwl7
Servidor backend escuchando en puerto 3001
Pool de conexiones MySQL inicializado.
~ $ # Logs del frontend (nginx sirviendo la app)
~ $ kubectl logs deploy/tienda-frontend -n tienda --tail=20
Found 2 pods, using pod/tienda-frontend-78df94cf8c-p8b2s
10.0.144.237 - - [28/Jun/2026:18:35:21 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:23 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:26 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:31 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:33 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:36 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:41 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:43 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:46 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:51 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:53 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:35:56 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:01 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:03 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:06 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:11 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:13 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:16 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:21 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
10.0.144.237 - - [28/Jun/2026:18:36:23 +0000] "GET / HTTP/1.1" 200 3662 "-" "kube-probe/1.35" "-"
~ $
```

Figura. Logs de la aplicación: backend escuchando y pool MySQL OK; frontend respondiendo 200 OK.

```
kubectl logs deploy/tienda-backend -n tienda --tail=40
kubectl logs deploy/tienda-frontend -n tienda --tail=20
```

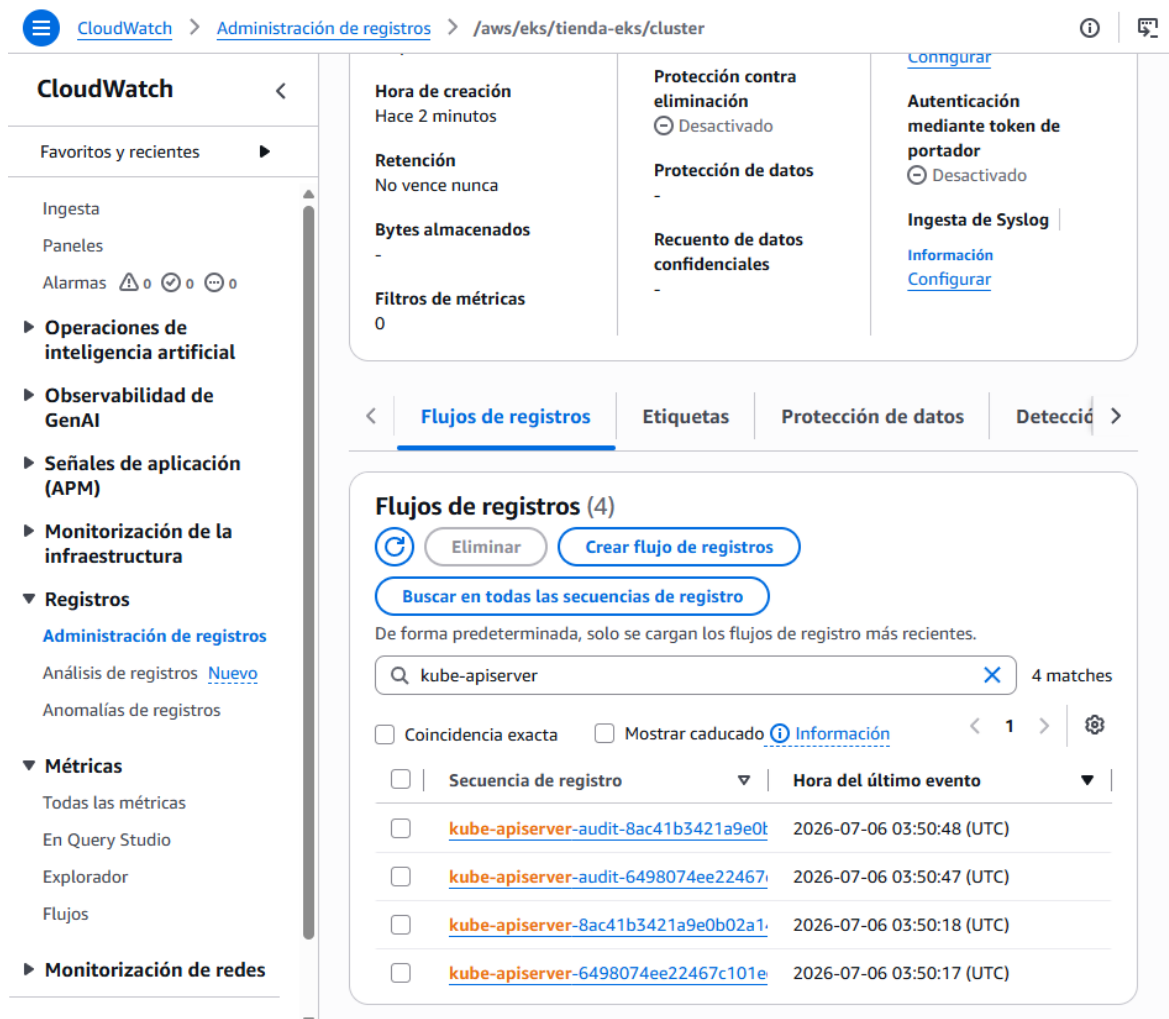


Figura. CloudWatch: log group /aws/eks/tienda-eks/cluster con streams del API Server y de Auditoría (kube-apiserver-audit).

Punto 8 · Seguridad básica (imágenes, escaneo, puertos, red y aislamiento de la BD)

La seguridad se aplica en varias capas (defensa en profundidad):

a) Imágenes minimalistas y non-root

Imágenes base alpine; backend con Dockerfile multietapa que deja fuera las devDependencies y corre como usuario node (sin privilegios de root). Menor superficie de ataque.

b) Escaneo de vulnerabilidades (Trivy — DevSecOps «shift-left»)

Entre el build y el push, el pipeline escanea las 3 imágenes con Trivy (de Aqua Security), filtrando severidades HIGH y CRITICAL: se detectan vulnerabilidades antes de publicar la imagen. Está configurado en modo informativo (exit-code 0): reporta en el log pero no bloquea el despliegue, porque las imágenes base (nginx, node, mysql) suelen traer CVEs que no dependen del equipo. En un entorno productivo real se elevaría a modo bloqueante (exit-code 1) — mejora futura.

c) Seguridad de red (grupos de seguridad y diseño privado)

No se crearon grupos de seguridad personalizados: EKS aprovisiona y gestiona automáticamente el del clúster (comunicación control plane ↔ nodos y nodo ↔ nodo) y el del ELB (abre el :80 y las reglas hacia el NodePort). La seguridad real viene del diseño de red: nodos en subredes privadas sin IP pública (inalcanzables desde Internet), salida solo por NAT, y exposición pública limitada al frontend (backend y base de datos internos). El endpoint del clúster público y privado mantiene el tráfico nodo↔API dentro de la VPC.

d) Aislamiento de la base de datos con NetworkPolicy (mejora implementada)

Por defecto la red de Kubernetes es plana: todo pod puede alcanzar a cualquier otro. Aunque la arquitectura de 3 capas hace que solo el backend hable con MySQL, a nivel de red nada impedía que el frontend (u otro pod comprometido) se conectara al puerto 3306. Para cerrar esa brecha se agregó la NetworkPolicy db-allow-backend-only (k8s/db-networkpolicy.yaml): aplica default-deny de ingreso sobre el pod tienda-db y solo permite conexiones al 3306 desde pods con la etiqueta app=tienda-backend. Así el acceso a la BD deja de ser una convención de diseño y pasa a ser una regla impuesta por el clúster.

Requisito en EKS: las NetworkPolicies solo surten efecto si el complemento Amazon VPC CNI tiene habilitado el enforcement («Enable Network Policy»); se dejó habilitado al configurar el add-on. Es una defensa complementaria al aislamiento por subredes: la red separa el clúster de Internet, y la NetworkPolicy segmenta el tráfico entre pods dentro del clúster.

e) Puertos mínimos

Solo el frontend se expone (ELB :80). Backend (ClusterIP :3001) y base de datos (headless :3306) son internos. SSH a los nodos deshabilitado.

Evidencia

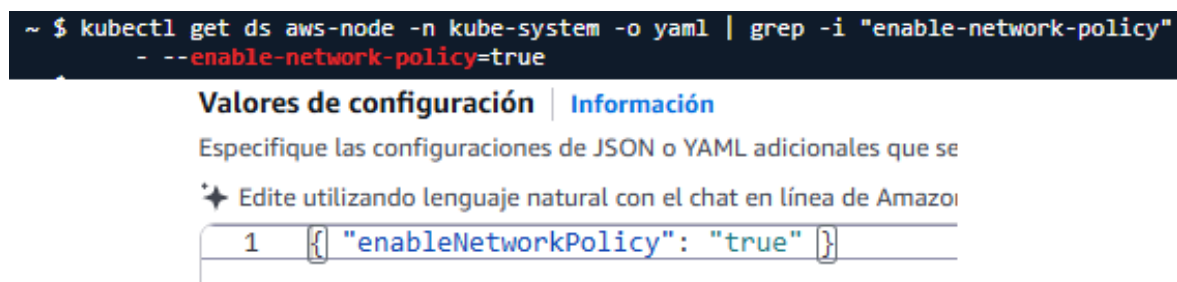


Figura. NetworkPolicy db-allow-backend-only activa y prueba de aislamiento: desde un pod sin la etiqueta de backend la conexión al 3306 es bloqueada; desde el backend, permitida.

```
kubectl apply -f k8s/db-networkpolicy.yaml
kubectl get networkpolicy -n tienda
# prueba de bloqueo (pod sin etiqueta backend -> falla):
```

```
kubectll run np-test -n tienda --image=busybox:1.28 --restart=Never --rm -i
t -- \
    sh -c 'nc -w 4 -z tienda-db 3306 && echo CONECTO || echo BLOQUEADO'
# prueba de acceso permitido (backend -> conecta):
kubectll exec -n tienda deploy/tienda-backend -- \
    sh -c 'nc -w 4 -z tienda-db 3306 && echo PERMITIDO'
```

Punto 9 · Orquestación y escalabilidad (por qué EKS y cómo escala)

Por qué un orquestador (EKS) y no despliegue manual

Un orquestador aporta lo que un despliegue manual (contenedores sueltos en una EC2) no da: autorrecuperación (recrea pods caídos), balanceo y descubrimiento de servicios, despliegues sin caída (rolling updates) y autoescalado. Se eligió EKS (Kubernetes gestionado) en lugar de ECS (orquestador propio de AWS) por ser el estándar de la industria y portable. ECR acompaña a EKS como registro de imágenes; ECS no se utiliza.

Autoescalado horizontal (HPA)

Se configuró HPA para ambos servicios visibles: backend 2→10 réplicas con umbral 70% de CPU, y frontend 2→6 réplicas con umbral 60%. El requisito previo es el Metrics Server (complemento gestionado del clúster). El HPA calcula el % de CPU contra el request de cada Deployment (backend request=100m / límite=500m), por lo que el umbral de 70% equivale a 70m de CPU. Sin requests/limits no habría métrica y el HPA quedaría en <unknown>.

Justificación del umbral (70% backend / 60% frontend): se deja margen antes de saturar el pod para que el HPA tenga tiempo de crear réplicas nuevas (que tardan en arrancar) antes de degradar el servicio. El frontend usa un umbral menor por ser la cara visible al usuario: conviene anticiparse más.

Prueba de carga y escalado

Se generó carga artificial sobre el backend con un pod load-generator (bucles en paralelo de peticiones a /api/health). Resultado: el CPU subió sobre 400%, el HPA escaló 2 → 4 → 8 → 10 réplicas (tope configurado) y, al repartirse la carga, el CPU promedio bajó del umbral; al retirar la carga, volvió a 2 réplicas tras el enfriamiento (~3–5 min). Concepto clave: el HPA escala pods, no nodos; esos pods nuevos nacen sobre los mismos 2 nodos.

Autorrecuperación y alta disponibilidad: al borrar un pod del backend, el clúster crea automáticamente uno nuevo manteniendo las 2 réplicas. Los nodos se reparten en 2 AZ (un único node group con Auto Scaling Group), lo que da tolerancia a la caída de una zona.

Evidencia

```
~ $ kubectl get apiservices v1beta1.metrics.k8s.io
NAME                SERVICE          AVAILABLE   AGE
v1beta1.metrics.k8s.io  kube-system/metrics-server  True        107m

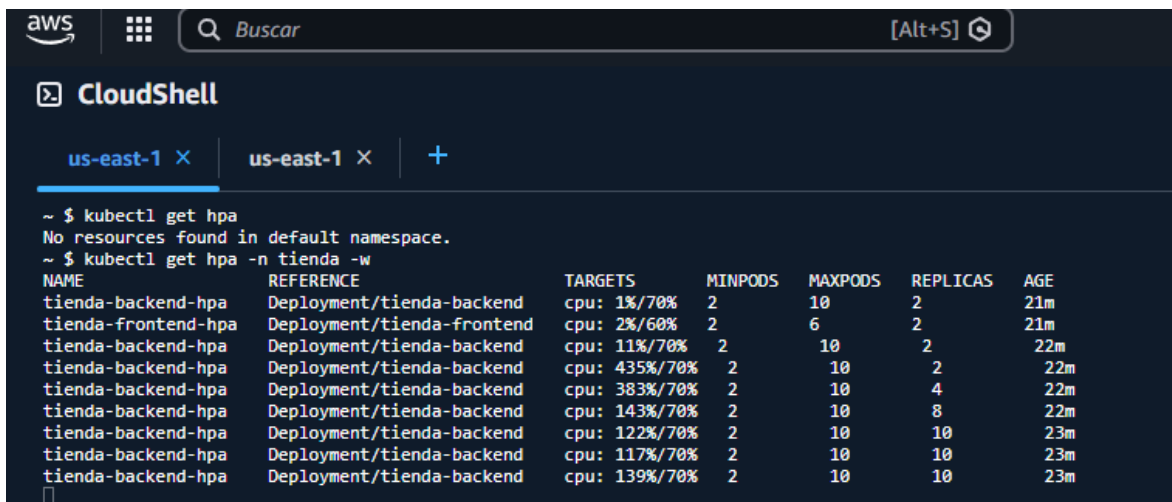
~ $ kubectl top pods -n tienda
NAME                                CPU(cores)   MEMORY(bytes)
tienda-backend-848b954bd5-9ccrb    1m           21Mi
tienda-backend-848b954bd5-s2mzp    1m           21Mi
tienda-db-654f8b9d68-r9p5l        7m           423Mi
tienda-frontend-78df94cf8c-p8b2s   1m           3Mi
tienda-frontend-78df94cf8c-x6gqv   1m           3Mi

~ $ kubectl get hpa -n tienda
NAME                REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
tienda-backend-hpa  Deployment/tienda-backend  cpu: 1%/70%    2         10        2          16m
tienda-frontend-hpa  Deployment/tienda-frontend  cpu: 2%/60%    2         6         2          16m

~ $
```

Figura. Metrics API disponible: HPA con % real de CPU y kubectl top pods funcionando.

```
kubectl get apiservices v1beta1.metrics.k8s.io
kubectl top pods -n tienda
kubectl get hpa -n tienda
```



The screenshot shows the AWS CloudShell interface with the terminal output of HPA scaling. The terminal shows the following commands and output:

```
~ $ kubectl get hpa
No resources found in default namespace.
~ $ kubectl get hpa -n tienda -w
NAME                REFERENCE          TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
tienda-backend-hpa  Deployment/tienda-backend  cpu: 11%/70%    2         10        2          22m
tienda-backend-hpa  Deployment/tienda-backend  cpu: 435%/70%   2         10        2          22m
tienda-backend-hpa  Deployment/tienda-backend  cpu: 383%/70%   2         10        4          22m
tienda-backend-hpa  Deployment/tienda-backend  cpu: 143%/70%   2         10        8          22m
tienda-backend-hpa  Deployment/tienda-backend  cpu: 122%/70%   2         10        10         23m
tienda-backend-hpa  Deployment/tienda-backend  cpu: 117%/70%   2         10        10         23m
tienda-backend-hpa  Deployment/tienda-backend  cpu: 139%/70%   2         10        10         23m
```

Figura. HPA escalando: CPU por sobre el umbral y número de réplicas subiendo (2→4→8).


```
CloudShell
us-east-1 X us-east-1 X +

~ $ kubectl run -n tienda load-generator --image=busybox:1.28 --restart=Never -- \
> /bin/sh -c "for i in 1 2 3 4 5 6 7 8 9 10; do (while true; do \
> wget -q -O- http://tienda-backend:3001/api/health >/dev/null 2>&1; done) & done; wait"
pod/load-generator created
~ $
```

CloudShell

us-east-1 X us-east-1 X +

Every 2.0s: kubectl get hpa,pods -n tienda -o wide

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/tienda-backend-hpa	Deployment/tienda-backend	cpu: 1%/70%	2	10	2	143m
horizontalpodautoscaler.autoscaling/tienda-frontend-hpa	Deployment/tienda-frontend	cpu: 2%/60%	2	6	2	143m

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/load-generator	1/1	Running	0	2s	10.0.138.13	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-54zs7	1/1	Running	0	16m	10.0.184.127	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-szg5l	1/1	Running	0	16m	10.0.137.104	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-db-9d44cf99-jkh4t	1/1	Running	0	16m	10.0.139.118	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-65ccp	1/1	Running	0	15m	10.0.140.40	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-pc6nz	1/1	Running	0	15m	10.0.186.121	ip-10-0-181-187.ec2.internal	<none>	<none>

CloudShell

us-east-1 X us-east-1 X +

Every 2.0s: kubectl get hpa,pods -n tienda -o wide

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
horizontalpodautoscaler.autoscaling/tienda-backend-hpa	Deployment/tienda-backend	cpu: 77%/70%	2	10	10	144m
horizontalpodautoscaler.autoscaling/tienda-frontend-hpa	Deployment/tienda-frontend	cpu: 2%/60%	2	6	2	144m

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
pod/load-generator	1/1	Running	0	69s	10.0.138.13	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-4bkbw	1/1	Running	0	47s	10.0.133.178	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-54zs7	1/1	Running	0	17m	10.0.184.127	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-5dtb8	1/1	Running	0	17s	10.0.181.134	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-6z6vt	1/1	Running	0	17s	10.0.133.92	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-b558g	1/1	Running	0	32s	10.0.183.242	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-dsvbc	1/1	Running	0	17s	10.0.185.68	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-gvsln	1/1	Running	0	17s	10.0.140.157	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-svv2p	1/1	Running	0	32s	10.0.133.70	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-szg5l	1/1	Running	0	17m	10.0.137.104	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-backend-78f84ccd66-txxzc	1/1	Running	0	32s	10.0.186.248	ip-10-0-181-187.ec2.internal	<none>	<none>
pod/tienda-db-9d44cf99-jkh4t	1/1	Running	0	17m	10.0.139.118	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-65ccp	1/1	Running	0	17m	10.0.140.40	ip-10-0-131-186.ec2.internal	<none>	<none>
pod/tienda-frontend-664d8f654-pc6nz	1/1	Running	0	16m	10.0.186.121	ip-10-0-181-187.ec2.internal	<none>	<none>

Figura. Backend en carga (~77%/70%) con ~10 pods Running repartidos en ambos nodos.

```
kubectl run -n tienda load-generator --image=busybox:1.28 --restart=Never -- \
/bin/sh -c "for i in 1 2 3 4 5 6 7 8 9 10; do (while true; do
  wget -q -O- http://tienda-backend:3001/api/health >/dev/null 2>&1; done)
& done; wait"
watch -n 2 "kubectl get hpa,pods -n tienda -o wide"
```

CloudShell

us-east-1 X us-east-1 X us-east-1 X us-east-1 X +

```
~ $ kubectl delete pod load-generator -n tienda
pod "load-generator" deleted from tienda namespace
~ $ kubectl get hpa -n tienda -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
tienda-backend-hpa	Deployment/tienda-backend	cpu: 1%/70%	2	10	2	154m
tienda-frontend-hpa	Deployment/tienda-frontend	cpu: 2%/60%	2	6	2	154m

Figura. Scale-down: tras retirar la carga el HPA reduce las réplicas de vuelta a 2.

```

us-east-1 X us-east-1 X +
~ $ kubectl get pods -n tienda
NAME                                READY   STATUS    RESTARTS   AGE
tienda-backend-848b954bd5-9ccrb    1/1     Running   0           34m
tienda-backend-848b954bd5-vcwl7    1/1     Running   0           11m
tienda-db-654f8b9d68-r9p5l         1/1     Running   0           34m
tienda-frontend-78df94cf8c-p8b2s   1/1     Running   0           34m
tienda-frontend-78df94cf8c-x6gqv   1/1     Running   0           34m
~ $ kubectl delete pod tienda-backend-848b954bd5-9ccrb -n tienda
pod "tienda-backend-848b954bd5-9ccrb" deleted from tienda namespace

```

```

~ $ kubectl get pods -n tienda
NAME                                READY   STATUS    RESTARTS   AGE
tienda-backend-848b954bd5-4fr7x    1/1     Running   0           15s
tienda-backend-848b954bd5-9ccrb    1/1     Terminating   0           35m
tienda-backend-848b954bd5-vcwl7    1/1     Running   0           12m
tienda-db-654f8b9d68-r9p5l         1/1     Running   0           36m
tienda-frontend-78df94cf8c-p8b2s   1/1     Running   0           35m
tienda-frontend-78df94cf8c-x6gqv   1/1     Running   0           35m
~ $

```

Figura. Autorrecuperación: al eliminar un pod del backend, Kubernetes recrea otro automáticamente.

```

kubectl delete pod -n tienda
kubectl get pods -n tienda # aparece el nuevo pod recreado

```

Mejora futura: un Cluster Autoscaler escalaría también los nodos EC2 cuando el HPA pida más pods de los que caben. No se implementó (limitación del laboratorio), pero queda documentado. Hoy el HPA es dinámico y el número de nodos es fijo (2–4).

Mejoras aplicadas y mejora continua

Sobre la solución se aplicaron mejoras de estructura, contenerización, automatización y seguridad:

- **Desarrollo local:** se incorporó docker-compose.yml (3 servicios, red interna, healthcheck de MySQL, volumen y depends_on), con los mismos nombres DNS que en Kubernetes para que el mismo artefacto corra en local y en la nube.
- **Contenedores:** backend con Dockerfile multietapa (deps → runtime) y usuario no-root (USER node); .dockerignore en los 3 componentes; imágenes minimalistas.
- **Calidad (CI):** etapa de test en el pipeline (Jest + supertest con la BD mockeada) como compuerta previa al despliegue (needs: test).
- **Seguridad de imágenes:** escaneo de vulnerabilidades con Trivy (HIGH/CRITICAL, informativo) entre build y push (shift-left DevSecOps).

- **Seguridad de red intra-clúster:** NetworkPolicy db-allow-backend-only que aísla la base de datos (solo el backend alcanza el 3306), con el enforcement de Network Policy habilitado en el add-on VPC CNI.
- **CI/CD:** pipeline que usa el Environment production, genera el Secret de MySQL, despliega también la imagen de la DB y etiqueta imágenes con SHA + latest; el Metrics Server no se instala desde el pipeline (solo se verifica), eliminando el conflicto que dejaba el HPA en <unknown>.

Problemas encontrados y limitaciones

Problema	Solución
HPA en «unknown» y kubectl top sin métricas.	Conflicto entre el metrics-server de upstream y el complemento del EKS; se quitó la instalación del pipeline y se dejó solo el complemento gestionado, con una verificación de la Metrics API.
El job build-and-deploy fallaba al resolver el escaneo Trivy.	La versión fijada del trivy-action no existía; se apuntó a la referencia vigente (aquasecurity/trivy-action@master).
Credenciales del laboratorio que rotan cada sesión.	Secrets en GitHub Environment + aws eks update-kubeconfig al inicio de cada sesión.
type: LoadBalancer requería AWS LB Controller (no disponible).	Se usó un Classic ELB (Service LoadBalancer sin anotaciones del controller).
Riesgo de acceso directo frontend→BD a nivel de red (red plana de Kubernetes).	NetworkPolicy db-allow-backend-only + enforcement del add-on VPC CNI.

Limitación conocida: persistencia efímera de la base de datos

El Deployment de MySQL (k8s/mysql-deployment.yaml) monta un volumen emptyDir en /var/lib/mysql. Un emptyDir está atado al ciclo de vida del pod, por lo que la base de datos es efímera: si el pod tienda-db se borra, se reprograma o se realiza un redeploy/rollout (kubectl set image), el volumen se destruye y MySQL se reinicializa desde cero, conservando solo los productos «semilla» de la imagen y perdiendo los cambios hechos por CRUD en tiempo de ejecución. Como el pipeline aplica set image también a la DB en cada push, el CRUD debe demostrarse después del último despliegue. Esta decisión es intencional dado el contexto de laboratorio (se priorizó la simplicidad y el ahorro de créditos); su superación se detalla en la Conclusión.

5. Conclusión

La solución implementada cumple con los objetivos de negocio de Tienda Perritos y con los nueve puntos del encargo. La aplicación quedó contenedorizada para desarrollo local con Docker Compose y orquestada en producción sobre AWS EKS, en una arquitectura de producción (2 AZ, nodos privados, ELB público) que aporta alta

disponibilidad y seguridad por defensa en capas.

La automatización del ciclo de vida con GitHub Actions eliminó los despliegues manuales: un commit a main ejecuta pruebas automáticas, escanea las imágenes, reconstruye, publica y despliega la aplicación en ~2 minutos, con recuperación sin downtime gracias a los rollouts de Kubernetes. La escalabilidad quedó demostrada con el HPA, que escaló el backend de 2 a 10 réplicas ante una carga real y volvió a su estado base al cesar la demanda. La seguridad se abordó sacando todo secreto del código (GitHub Environment), escaneando imágenes con Trivy y aislando la base de datos con una NetworkPolicy.

En cuanto a mejora continua, la prioridad es dotar de persistencia a la base de datos, hoy efímera por el uso de emptyDir. La recomendación es reemplazar ese volumen por un PersistentVolumeClaim (PVC) respaldado por Amazon EBS (addon aws-ebs-csi-driver) y, idealmente, migrar el Deployment de MySQL a un StatefulSet; como alternativa de producción, desacoplar la base de datos empleando un servicio gestionado como Amazon RDS for MySQL. Otros pasos naturales: incorporar un Cluster Autoscaler para escalar los nodos EC2; elevar el escaneo Trivy a modo bloqueante; añadir HTTPS/Ingress con certificado ACM y dominio propio en Route 53; y desplegar un NAT Gateway por AZ para eliminar el punto único de fallo. En conjunto, el proyecto evidencia competencias integrales de DevOps: automatizar, desplegar, asegurar y optimizar software en un entorno de nube real aplicando buenas prácticas profesionales.